Trajectory-exact constrained decoding for masked diffusion models: routes, twists and ambiguity

Working notes

October 2026

Abstract

These notes describe a decoder that samples a masked diffusion language model conditioned on a hard constraint, in the sense of drawing from the model's own output distribution restricted to the constraint. The constraint can be a regular expression, a counting condition, a copy relation, a context-free grammar (ambiguous grammars included) or a tree structure. The decoder has two layers. The first is a *route*: a tractable structure that computes the probability of the constraint under a product distribution and samples from that distribution restricted to the constraint. The second is twisted sequential Monte Carlo, which corrects the error that per-step constrained decoding makes when the denoiser revises its predictions after seeing its own outputs. We collect the mathematical results and algorithms behind both layers and the open directions. Results taken from the literature are stated with a reference to their proofs. Results that are new here, or new combinations of known pieces, are proved. Experimental results are not included.

Contents

1 Introduction **2**
- 1.1 Masked diffusion sampling . . . 2
- 1.2 What conditional sampling asks for . . . 2
- 1.3 The approach in these notes . . . 3
- 1.4 What is known and what is new . . . 4
- 1.5 Outline . . . 4

2 Background: twisted sequential Monte Carlo **4**

3 Setup and notation **5**

4 Step-level theory **5**
- 4.1 The constraint polynomial . . . 5
- 4.2 Semirings and ambiguity . . . 6
- 4.3 When is the step-level problem tractable? . . . 6

5 Routes and their exact samplers **7**
- 5.1 Automata . . . 7
- 5.2 Grammar charts . . . 8
- 5.3 Factor graphs . . . 8

5.4 Determinants . . . 9
5.5 From classes to tokens . . . 9
5.6 Route compiler . . . 9

6 Trajectory-exact decoding with any route **9**
6.1 The step-exact proposal . . . 9
6.2 The algorithm . . . 10
6.3 Correctness . . . 10
6.4 Choosing the twist separately from the proposal . . . 11
6.5 A two-sided identity . . . 12

7 Resampling with an over-confident twist **12**
7.1 Continuation values and hidden mass . . . 12
7.2 Defensive resampling . . . 13
7.3 Tempered intermediate twists . . . 14

8 Ambiguity **14**

9 Beyond exact routes **15**
9.1 A regular superset with Boolean pruning [ours] . . . 15
9.2 Conjunctions . . . 16
9.3 The intractable boundary . . . 16

10 Open directions **16**

11 Summary of results **17**

1 Introduction

1.1 Masked diffusion sampling

A masked diffusion language model (MDLM) [11] produces a sequence of length L by starting from a sequence of mask symbols and filling in positions over T steps. At each step a neural *denoiser* looks at the current partial sequence and outputs, for every masked position, a categorical distribution over the vocabulary. A *scheduler* picks which positions to reveal at this step, and those positions are filled by sampling from the denoiser's distributions, independently across positions. The revealed positions stay fixed and the process repeats until nothing is masked. We call this procedure the *native sampler*, and write p for the distribution it induces over complete sequences (and over whole trajectories, when that matters).

Two features of this sampler shape everything below. Within a step, the denoiser's predictions are a product distribution over positions. Across steps, those predictions change, because each call of the denoiser conditions on everything revealed so far.

1.2 What conditional sampling asks for

Let C be a set of acceptable outputs: strings matching a pattern, balanced bracket sequences, RNA sequences that can fold into a valid secondary structure, and so on. The goal is to sample from

$$p(x \mid x \in C) = \frac{p(x)\,\mathbf{1}_C(x)}{p(C)},$$

the model's own distribution restricted to C. Rejection sampling (run the native sampler until the output lies in C) achieves this exactly but is useless when $p(C)$ is small. Two cheaper methods guarantee $x \in C$. Each changes the distribution.

Token masking forbids, at each step, tokens that would make C unreachable, and renormalises over the rest. It ignores how much of the model's probability stays inside C after each choice, so it over-represents sequences whose early tokens leave little valid probability mass.

Step-exact decoding [3] goes further. At each step it computes the product distribution given by the denoiser's current predictions, restricts it to C, samples a complete sequence from that restricted distribution, and keeps the positions the scheduler chose to reveal. Each step is then exact for the product distribution the denoiser predicts at that moment. The composition over steps is still biased, as the following example shows.

Example 1.1 (Trajectory bias with two positions)**.** Let $L = 2$, $T = 2$, reveal position 1 at the first step and position 2 at the second, and let $C = \{(a, a) : a \in V\}$, i.e. the two tokens must agree. Write $p^1(a)$ for the denoiser's prediction at position 1 from the all-mask state, $p^2(a \mid \varnothing)$ for its prediction at position 2 from the same state, and $p^2(a \mid a)$ for its prediction at position 2 after position 1 has been revealed as a. The native conditional is

$$p(x = (a, a) \mid C) \;\propto\; p^1(a)\, p^2(a \mid a).$$

Step-exact decoding draws position 1 from the restricted product at the first step, which gives a with probability $\propto p^1(a)\, p^2(a \mid \varnothing)$; the second step is then forced. The two laws agree only if $p^2(a \mid a)/p^2(a \mid \varnothing)$ does not depend on a, i.e. only if revealing position 1 does not change the relative predictions at position 2. For a real language model it almost always does.

The ratio $p^2(a \mid a)/p^2(a \mid \varnothing)$ in Example 1.1 is exactly the importance weight that corrects the error. It compares the probability of the constraint under the *updated* predictions with its probability under the *old* predictions, for the same partial sequence. TWISTER [13] applies this correction at every step and resamples a population of particles, using twisted sequential Monte Carlo. It proves that the result targets the native conditional exactly, for constraints given by deterministic finite automata (DFAs).

1.3 The approach in these notes

We separate the two concerns.

1. A *route* handles everything that happens within one step. Given per-position weights, it computes the probability of C under the product distribution, samples from the product distribution restricted to C, and checks membership in C. Automata are one kind of route. Grammar charts, factor graphs and determinants are others, and each covers constraint families the others cannot handle efficiently.
2. A *trajectory layer* handles what happens across steps. It is TWISTER's twisted SMC with the automaton replaced by any route.

The main theoretical observation (Theorem 6.2) is that the trajectory layer needs very little from the route. The route's sampler must draw from the same measure whose total its mass computation returns, and the last step must check membership in C exactly. The route does not have to be unambiguous. An ambiguous grammar, which counts a string once per parse tree, gives the right answer as the number of particles grows. Ambiguity only increases the variance.

The other main result concerns resampling (Section 7). The SMC estimate of $p(C)$ is unbiased for every number of particles. When the denoiser's predictions at some intermediate step badly

understate how likely the constraint is to be met in the end, adaptive resampling discards particles that would later have recovered. A non-trivial fraction of the estimate's expectation can then sit in events too rare to observe. We characterise this hidden mass exactly and give two remedies that keep the estimate unbiased and have guarantees that do not depend on the quality of the predictions.

1.4 What is known and what is new

Much of the step-level theory is known in the probabilistic-circuits literature, often in more general algebraic form. Section 4 states it in our notation with references. The route-generic trajectory correction, the analysis of resampling, the remedies, the ambiguity bound and the superset construction are new here, as far as we know. Each result is tagged [background] (known, cited, proof referenced), [ours] (proved here) or [proposed] (a direction, not a result). Section 11 lists all of them.

1.5 Outline

Section 2 reviews the parts of sequential Monte Carlo we use. Section 3 fixes notation. Section 4 gives the step-level theory and Section 5 the routes and their samplers. Section 6 contains the trajectory-exact decoder and its correctness theorem. Section 7 analyses resampling, Section 8 ambiguity, and Section 9 constraints that need more than an exact route. Section 10 lists open directions.

2 Background: twisted sequential Monte Carlo

This section collects standard facts; [4] is the reference for all of them.

Importance sampling on paths. Let γ be an unnormalised target on trajectories $x_{0:T}$ with normaliser $\mathcal{Z} = \sum \gamma$, and let q be a proposal with $q > 0$ wherever $\gamma > 0$. The weight $W = \gamma/q$ satisfies $\mathbb{E}_q[W] = \mathcal{Z}$, and the weighted samples $\{(x^k, W^k)\}$ estimate expectations under $\gamma/\mathcal{Z}$. Here $\gamma(x_{0:T}) = p(x_{0:T})\mathbf{1}_C(x_T)$, so $\mathcal{Z} = p(C)$, which we write h_0.

Sequential form and twists. If p and q factorise over steps, $p = \prod_t \kappa_{t+1}$ and $q = \prod_t M_{t+1}$, the weight can be accumulated step by step. A *twist* is a sequence of non-negative functions $\psi_0, \dots, \psi_T$ with $\psi_T = \mathbf{1}_C$. It defines intermediate targets $\gamma_t(x_{0:t}) = p(x_{0:t})\psi_t(x_t)$ and *potentials*

$$G_{t+1}(x_t, x_{t+1}) = \frac{\kappa_{t+1}(x_{t+1} \mid x_t)\, \psi_{t+1}(x_{t+1})}{M_{t+1}(x_{t+1} \mid x_t)\, \psi_t(x_t)}. \tag{1}$$

The product $\psi_0(x_0) \prod_t G_{t+1}$ telescopes to γ/q, so the final weight is unchanged by the choice of twist. The twist only changes the intermediate weights, and therefore which particles survive resampling. The ideal twist is the *Doob function* $h_t(x) = \Pr_p(X_T \in C \mid X_t = x)$. With $\psi_t = h_t$, the intermediate weights reflect exactly how likely each particle is to end in C.

Resampling and the normalising-constant estimate. With K particles, the weights tend to concentrate on a few of them. A common diagnostic is the effective sample size $\mathrm{ESS} = (\sum_k W^k)^2 / \sum_k (W^k)^2 \in [1, K]$. Adaptive resampling draws K new particles from the current

ones in proportion to their weights whenever ESS falls below a threshold τK, and then resets the weights. The estimate

$$\widehat{Z} = \prod_{\text{resampling times}} \Big(\frac{1}{K}\sum_k W^k\Big) \times \Big(\frac{1}{K}\sum_k W^k\Big)_{\text{final}}$$

is unbiased for $\mathcal{Z}$ at every K [4]. The weighted particles are consistent for the normalised target as $K \to \infty$.

3 Setup and notation

Tokens and classes. Let V be the vocabulary and $\mathtt{m} \notin V$ the mask symbol. Constraints are usually stated over a small alphabet Σ of token *classes*, through a map $\phi : V \to \Sigma$ (digit, opening bracket, nucleotide). We write $C \subseteq \Sigma^L$, and a token sequence x satisfies C if $\phi(x) \in C$. For a partial sequence $x \in (V \cup \{\mathtt{m}\})^L$, $\mathcal{M}(x)$ is the set of its masked positions.

Native kernel. Following [13], one native step from x to x' has probability

$$\kappa_{t+1}(x' \mid x) = s_{t+1}(r \mid x) \prod_{i \in r} p_\theta^i(x'_i \mid x), \tag{2}$$

where $r \subseteq \mathcal{M}(x)$ is the set of positions revealed at the step, s_{t+1} is the scheduler (for example, L/T positions chosen uniformly among the masked ones) and $p_\theta^i(\cdot \mid x)$ is the denoiser's prediction at position i. The native path law is $p(x_{0:T}) = \prod_t \kappa_{t+1}(x_{t+1} \mid x_t)$, with $x_0 = \mathtt{m}^L$ and x_T fully revealed.

Target. The *Doob path law* $p^\star(x_{0:T}) \propto p(x_{0:T})\mathbf{1}_C(x_T)$ has terminal marginal $p(\cdot \mid C)$. It is a Markov chain with kernel $\kappa_{t+1}h_{t+1}/h_t$. The Doob function h_t depends on the denoiser's outputs at every future state, so it cannot be computed directly.

Frozen weights and the clamped sum. For a partial sequence x, the class weights are

$$w_i(\sigma \mid x) = \sum_{v:\, \phi(v)=\sigma} p_\theta^i(v \mid x) \qquad (i \in \mathcal{M}(x)),$$

and revealed positions are clamped to their class, $w_i(\sigma \mid x) = [\sigma = \phi(x_i)]$. The *clamped sum* of a partial sequence x' under the predictions made at x is

$$Z(x'; x) = \sum_{y \in C} \prod_{i \in \mathcal{M}(x')} w_i(y_i \mid x) \prod_{i \notin \mathcal{M}(x')} [y_i = \phi(x'_i)]. \tag{3}$$

$\eta_t(x) := Z(x; x)$ is the probability that C holds if every masked position is filled independently from the current predictions. TWISTER uses η_t in place of h_t. We call it the *frozen twist*, because it treats the current predictions as if they would not change.

4 Step-level theory

4.1 The constraint polynomial

Every per-step quantity in constrained decoding is a query on a single polynomial.

Definition 4.1 (Constraint polynomial)**.** For $C \subseteq \Sigma^L$ and $w \in \mathbb{R}_{\geq 0}^{L \times \Sigma}$, $P_C(w) = \sum_{y \in C} \prod_{i=1}^{L} w_{i,y_i}$.

The frozen twist is $\eta_t(x) = P_C(w(x))$. The constrained marginals are $\mu_{i,\sigma} = \Pr_q(y_i = \sigma \mid C) = w_{i,\sigma}\, \partial_{i,\sigma} P_C(w)/P_C(w)$, and step-exact decoding needs samples from $q(y \mid C) \propto \prod_i w_{i,y_i} \mathbf{1}_C(y)$. P_C is set-multilinear (each monomial contains one variable per position), so the marginal formula holds for any circuit that computes P_C.

Definition 4.2 (Structure circuit and derivations)**.** A *structure circuit* is a smooth, decomposable, monotone arithmetic circuit over the leaves $w_{i,\sigma}$ with unit parameters. Automaton trellises, inside charts of grammars restricted to length L, and variable elimination with 0/1 factors are all of this kind. A structure circuit S computes

$$P_S(w) = \sum_{y \in \Sigma^L} d_S(y) \prod_i w_{i,y_i},$$

where $d_S(y) \in \mathbb{N}$ is the number of *derivations* of y in S (accepting paths of an automaton, parse trees of a grammar). S represents C if $\operatorname{supp} d_S = C$, and is *unambiguous* if $d_S \equiv 1$ on C.

4.2 Semirings and ambiguity

Proposition 4.3 ([background])**.** *Let S represent C. Evaluated on leaves $w \geq 0$:*

semiring	*value of S*	*equals the constrained query*
Boolean $(\vee, \wedge)$	$[\exists y \in C$ *consistent with* $w]$	*always (completability)*
max-times	$\max_{y \in C} \prod_i w_{i,y_i}$	*always (MAP, recovered by backtracking)*
sum-product	$\sum_y d_S(y) \prod_i w_{i,y_i}$	*if and only if $d_S \equiv 1$ on C*

Proof. A special case of Wang et al. [14], Thm. 4 and Ex. 2: mapping a Boolean circuit into a semiring is tractable when the semiring is idempotent or the circuit is deterministic.

Every derivation of y has weight $\prod_i w_{i,y_i}$. Idempotent addition counts the $d_S(y)$ copies once, and ordinary addition counts all of them. Top-down sampling from a smooth, decomposable, monotone circuit is correct for the polynomial the circuit computes, deterministic or not. Ambiguity therefore makes a sampler draw from the derivation-weighted distribution $\propto d_S(y) \prod_i w_{i,y_i}$. Dang and Ermon call their NFA variant a "soft proxy" for this reason [3].

Corollary 4.4 (Exact MAP for any CFG [background])**.** *For every context-free grammar G, including inherently ambiguous ones, Viterbi on the length-L chart computes* $\arg\max_{y \in L(G) \cap \Sigma^L} \prod_i w_{i,y_i}$ *in $O(L^3|G|)$.*

Proof. The max-times row of Proposition 4.3 applied to the chart of Section 5.2.

4.3 When is the step-level problem tractable?

Theorem 4.5 ([background])**.** *Fix Σ and a family $\{C_L\}$. With all bounds polynomial in L and $|\Sigma|$, the following are equivalent.*

(a) P_{C_L} has a polynomial-size arithmetic circuit over $\mathbb{R}$. Subtraction is allowed; the circuit need not be decomposable, deterministic or monotone.
(b) A real-RAM algorithm computes $P_{C_L}(w)$ for every $w \geq 0$.
(c) A real-RAM algorithm computes all constrained marginals $\mu_{i,\sigma}(w)$ whenever $P_{C_L}(w) > 0$.

Given a circuit of size s, $P_C(w)$ costs $O(s)$, all $L|\Sigma|$ marginals together cost $O(s)$, and an exact sample from $q(\cdot \mid C)$ costs $O(Ls)$. If the circuit is smooth, decomposable and monotone, one top-down pass gives a sample.

Proof. (b)⇒(a) is the completeness theorem of Broadrick et al. [1], Prop. 5.4 (Sec. 5.1, proof in App. C). Their virtual-evidence marginalisation (Def. 5.1) is query (b) for $\mathbf{1}_C$ after one-hot encoding each position. Their proof uses Strassen's division elimination and a polynomial-identity argument on an open box; we checked that both steps go through for P_C with correctness only on the positive orthant. (a)⇒(b) is evaluation, and all marginals come from one reverse-mode differentiation pass. (c)⇒(b) and the $O(Ls)$ sampler are self-reducibility under clamping [7].

Broadrick et al. also show (their Prop. 4.9 and Thm. 4.10) that tractable marginals under hard evidence alone do not imply a small circuit. Condition (b) needs arbitrary weights, which decoding supplies, since the weights are the denoiser's predictions.

Remark 4.6 (Approximate sampling [background])**.** A polynomial-time almost-uniform sampler for $q(\cdot \mid C)$ that works for every clamped w yields an FPRAS for P_C [7]. An efficient sampler therefore implies efficient *approximate* counting, which is weaker than (a) to (c). The route compiler uses the converse to decide when to refuse a constraint (Section 9).

Remark 4.7 (Decomposable circuits do not cover everything [background])**.** Encode a tree on nodes $\{0, \dots, n\}$ by head pointers $y_m \in \{0, \dots, n\} \setminus \{m\}$, and let C be the set of arborescences rooted at 0. Then P_C is the directed spanning-tree polynomial. It has a polynomial-size circuit via the directed matrix-tree theorem (Section 5.4). Every monotone circuit for it has size $2^{\Omega(n)}$ [6]. Smooth decomposable circuits with non-negative parameters are monotone, so no automaton, grammar or bounded-treewidth route is polynomial for this constraint, while exact decoding through a determinant is.

5 Routes and their exact samplers

Definition 5.1 (Route)**.** A *route* for C provides three operations on weights $w \geq 0$:

- $\textsc{Mass}(w) = \sum_y d(y) \prod_i w_{i,y_i}$, for a fixed $d \geq 0$ with $\operatorname{supp} d \supseteq C$;
- $\textsc{Sample}(w)$, an exact draw from $y \propto d(y) \prod_i w_{i,y_i}$;
- $\textsc{Accepts}(y) = \mathbf{1}_C(y)$, an exact membership test.

The route is *exact* if $d = \mathbf{1}_C$, *derivation-weighted* if $d \in \mathbb{N}$ and $\operatorname{supp} d = C$, and a *superset route* if $\operatorname{supp} d \supsetneq C$.

The condition the trajectory layer relies on is that Sample draws from exactly the measure whose total Mass returns. The routes below all satisfy it. In each case the correctness of the mass computation and the sampler is standard; we give the recursions and references.

5.1 Automata

For a finite automaton (Q, q_0, δ, F) over Σ, the forward recursion

$$\alpha_0(q) = [q = q_0], \qquad \alpha_i(q) = \sum_{q', \sigma:\, q \in \delta(q', \sigma)} \alpha_{i-1}(q')\, w_{i,\sigma}, \qquad \textsc{Mass} = \sum_{q \in F} \alpha_L(q),$$

costs $O(L|\delta|)$. Sample is forward filtering, backward sampling. Draw $q_L \propto \alpha_L(q)[q \in F]$; then, for $i = L, \dots, 1$, draw (q_{i-1}, σ_i) with probability $\propto \alpha_{i-1}(q_{i-1}) w_{i,\sigma_i} [q_i \in \delta(q_{i-1}, \sigma_i)]$. For a DFA, $d = \mathbf{1}_C$. For an NFA, $d(y)$ is the number of accepting paths of y.

Constraints that compile to automata include forbidden and required motifs (via an Aho–Corasick automaton), counts such as "exactly k tokens of class c" or "a fraction within a band" (a counter automaton with $O(Lk)$ states in any variable order), regular expressions over classes, bounded-depth bracketing, sliding-window all-different constraints, and products of these.

5.2 Grammar charts

For a grammar in Chomsky normal form with nonterminals N, binary rules $A \to BC$ and lexical rules $A \to \sigma$, the inside algorithm on the chart of length L,

$$\beta(A, i, i{+}1) = \sum_{\sigma} [A \to \sigma]\, w_{i,\sigma}, \qquad \beta(A, i, j) = \sum_{A \to BC} \sum_{i<k<j} \beta(B, i, k)\, \beta(C, k, j),$$

$$\textsc{Mass} = \beta(S, 0, L),$$

costs $O(L^3|G|)$. The leaves for all positions come from one matrix product of the $L \times |V|$ denoiser output with the $|V| \times |N_{\text{pre}}|$ lexical map.

Algorithm 1 Top-down chart sampler

push $(S, 0, L)$
while the stack is non-empty **do**
    pop (A, i, j)
    if $j = i + 1$ **then** draw $\sigma \propto [A \to \sigma]\, w_{i,\sigma}$ and set $y_i \leftarrow \sigma$
    else draw $(A \to BC,\, k) \propto \beta(B, i, k)\, \beta(C, k, j)$; push (B, i, k) and (C, k, j)

Algorithm 1 draws each parse tree with probability proportional to its weight. The sampled string therefore has probability $\propto d(y) \prod_i w_{i,y_i}$, where $d(y)$ is its number of parse trees. On a GPU, all spans of one length are processed together and the draws are batched over particles.

Example 5.2 (Grammars used in the experiments)**.**

- Unambiguous Dyck-1, $S \to (S)S \mid \varepsilon$, converted to CNF.
- Ambiguous Dyck-1, $S \to SS \mid (S) \mid ()$. It generates the same language, but a string with several adjacent blocks has several parse trees.
- An RNA pairing grammar in the style of Nussinov and Jacobson [10]. Every base is paired, pairs do not cross, and each pair is one of AU, UA, GC, CG, GU, UG. Parse trees correspond one to one with secondary structures, so $d(y)$ is the number of structures y can fold into. This number can grow exponentially with L; $(GC)^n$ has Catalan(n) structures.

5.3 Factor graphs

If C is given by 0/1 factors and an elimination order has induced width ω, variable elimination computes $\textsc{Mass}$ in $O(L|\Sigma|^{\omega+1})$. $\textsc{Sample}$ eliminates in that order and samples in reverse. Derivations here are assignments, so $d = \mathbf{1}_C$. The case that motivates this route is the same-order copy relation $y_i = y_{i+k}$ for $i \le k$ [12], which has width 1:

$$\textsc{Mass} = \prod_{i \le k} \sum_{\sigma} w_{i,\sigma} w_{i+k,\sigma} \times \prod_{i > 2k} \sum_{\sigma} w_{i,\sigma}.$$

An automaton that reads positions in order needs $|\Sigma|^k$ states for the same relation.

5.4 Determinants

Let $w_m(h)$ be the weight that node m has head h. Let $\hat{L}$ be the $n \times n$ matrix on the non-root nodes with $\hat{L}_{mm} = \sum_{h \neq m} w_m(h)$ (the root included in the sum) and $\hat{L}_{hm} = -w_m(h)$ for $h \neq m$. The directed matrix-tree theorem gives $\textsc{Mass} = \det \hat{L}$. Differentiating $\log \det \hat{L}$ gives the marginals

$$\mu_m(h) = w_m(h)\big([\hat{L}^{-1}]_{mm} - [h \neq 0]\,[\hat{L}^{-1}]_{mh}\big)$$

(see [8]). Both cost $O(n^3)$. $\textsc{Sample}$ is Wilson's algorithm [15]: for each node not yet in the tree, run the random walk that moves from m to h with probability $w_m(h)/\sum_{h'} w_m(h')$ until it reaches the tree, erase the loops of the path and add the path to the tree. The resulting arborescence has probability $\propto \prod_m w_m(y_m)$. By Remark 4.7 no polynomial monotone route exists for this constraint.

5.5 From classes to tokens

A route draws a class sequence y. Each revealed position then draws a token within its class, $v \propto p_\theta^i(v \mid x)[\phi(v) = y_i]$. The combined token-level law is $\prod_i p_\theta^i(v_i \mid x)\mathbf{1}_C(\phi(v))/Z$, which is what the trajectory layer requires.

5.6 Route compiler

The compiler turns a constraint specification into a route. It estimates the cost of one pass for each candidate from compile-time statistics ($O(L|\delta|)$ for a chain, $O(L^3|G|)$ for a chart, $O(L|\Sigma|^{\omega+1})$ for variable elimination, $O(n^3)$ for a determinant) and picks the cheapest. Which route is cheapest depends on the representation and the variable order as well as on the constraint: copy is exponential as an in-order automaton and linear as a factor graph. When no candidate fits the budget, or the family has #P-hard counting (for example, CNF clauses over positions), the compiler refuses instead of approximating.

6 Trajectory-exact decoding with any route

6.1 The step-exact proposal

Write $Z_d(x'; x)$ for the clamped sum (3) with $\mathbf{1}_C$ replaced by the route's d, and $\eta_t(x) = Z_d(x; x)$.

Lemma 6.1 (Step-exact proposal [background])**.** *The kernel*

$$M_{t+1}(x' \mid x) = s_{t+1}(r \mid x) \prod_{i \in r} p_\theta^i(x'_i \mid x)\, \frac{Z_d(x'; x)}{Z_d(x; x)}$$

is a probability kernel. It is sampled by drawing r from the native scheduler, drawing a complete class sequence $y \sim \textsc{Sample}(w(x))$ with revealed positions clamped, drawing tokens within classes at the positions in r, and revealing only those positions.

Proof. TWISTER [13], Prop. 3.6, proof in App. A, stated for DFAs. The proof uses only the fact that the sampler draws from the measure whose total is $Z_d(x; x)$, so it applies to any route.

Combining Lemma 6.1 with (1), the scheduler and the token probabilities cancel, and the potential becomes

$$G_{t+1}(x, x') = \frac{Z_d(x; x)}{\psi_t(x)} \cdot \frac{\psi_{t+1}(x')}{Z_d(x'; x)}. \tag{4}$$

TWISTER takes $\psi_t = \eta_t$ for $t < T$, which gives $G_{t+1} = Z_d(x'; x')/Z_d(x'; x)$: the mass of the new partial sequence under the new predictions, divided by its mass under the old predictions. In Example 1.1 this ratio is $p^2(a \mid a)/p^2(a \mid \varnothing)$.

6.2 The algorithm

Algorithm 2 is TWISTER with the automaton replaced by a route. It has two optional parameters, introduced in Section 7: defensive mixing ϵ and twist power β. With $\epsilon = 0$ and $\beta = 1$ it is TWISTER's algorithm. With $K = 1$ and the weights ignored, it is step-exact decoding [3].

Algorithm 2 Route-generic twisted SMC (one run with K particles)

Require: route (Mass, Sample, Accepts); denoiser; T; K; ESS threshold τ; $\epsilon \in [0, 1)$; $\beta \in (0, 1]$

for each k: $x^k \leftarrow \mathtt{m}^L$, $\eta^k \leftarrow \textsc{Mass}(w(x^k))$, $\log W^k \leftarrow \beta \log \eta^k$; set $\log \widehat{Z} \leftarrow 0$
for $t = 0, \dots, T-1$ **do**
  for each particle k **do**
    draw $r \sim s_{t+1}(\cdot \mid x^k)$ and $y \sim \textsc{Sample}(w(x^k))$; draw tokens within classes on r; $x' \leftarrow x^k$ revealed on r
    $z \leftarrow Z_d(x'; x^k)$ ▷ mass of x' under the old predictions
    if $t < T-1$ **then** call the denoiser at x'; $\eta' \leftarrow Z_d(x'; x')$; $\psi' \leftarrow (\eta')^\beta$
    else $\psi' \leftarrow \textsc{Accepts}(x')$ ▷ exact membership at the last step
    $\log W^k \leftarrow \log W^k + (1-\beta)\log \eta^k + \log \psi' - \log z$; $x^k \leftarrow x'$; $\eta^k \leftarrow \eta'$
  if $t < T-1$ and $\mathrm{ESS}(W) < \tau K$ **then**
    $\log \widehat{Z} \leftarrow \log \widehat{Z} + \log \frac{1}{K} \sum_k W^k$; $\tilde{w} \leftarrow W / \sum_k W^k$; $\pi \leftarrow (1-\epsilon)\tilde{w} + \epsilon/K$
    draw ancestors $a_1, \dots, a_K$ from π (independently, or systematically); for each k: $x^k \leftarrow x^{a_k}$, $\eta^k \leftarrow \eta^{a_k}$, $W^k \leftarrow \tilde{w}_{a_k}/\pi_{a_k}$
$\log \widehat{Z} \leftarrow \log \widehat{Z} + \log \frac{1}{K} \sum_k W^k$
return weighted particles $\{(x^k, W^k)\}$ and $\widehat{Z}$

Per particle and step, the algorithm makes one denoiser call, one route sample and two mass evaluations: the clamped pass under the old predictions and the pass under the new ones. The latter is reused at the next step.

6.3 Correctness

Theorem 6.2 (Route-generic trajectory exactness [ours])**.** *Assume the denoiser gives every token positive probability. Let the route satisfy* $\operatorname{supp} d \supseteq C$*, with ambiguity allowed. Let the twists satisfy* $\psi_t(x) > 0$ *whenever* $h_t(x) > 0$*, and* $\psi_T = \mathbf{1}_C$*; for instance* $\psi_t = \eta_t^\beta$ *with any* $\beta > 0$*. Let* $W(x_{0:T}) = \psi_0(x_0) \prod_{t<T} G_{t+1}$ *and* $q = \prod_t M_{t+1}$*. Then:*

(i) $q(x_{0:T})\, W(x_{0:T}) = p(x_{0:T})\, \mathbf{1}_C(x_T)$ *for every path;*
(ii) the weighted particles of Algorithm 2 target the Doob path law $p^\star$ *and are consistent for* $p(\cdot \mid C)$ *as* $K \to \infty$*;*
(iii) $\mathbb{E}[\widehat{Z}] = h_0$ *for every* $K \geq 1$*, every* $\epsilon \in [0, 1)$*, and every resampling rule whose decision at each step depends only on the current particles and weights;*
(iv) the derivation count affects only the variance: the factor $d(x_T)$ *introduced by the last proposal step cancels in the last potential.*

Proof. (i) By Lemma 6.1, $q = \prod_t \kappa_{t+1} Z_d(x_{t+1}; x_t)/Z_d(x_t; x_t)$. Multiplying by (4), the factors Z_d cancel, and the twists telescope to $qW = \psi_0 \prod_t \kappa_{t+1}\psi_{t+1}/\psi_t = p\,\psi_T = p\,\mathbf{1}_C$. The divisions are

legitimate on the relevant paths. If x' can still be completed to an element of C, some $y \in C$ agrees with x'; then $d(y) > 0$, so $Z_d(x'; x) > 0$ under the positivity assumption, and $M_{t+1}(x' \mid x) > 0$. If $\psi_t(x) = 0$, the particle already has weight zero, and we set $0/0 = 0$.

(ii) follows from (i) by the standard Feynman–Kac argument [4].

(iii) Unbiasedness of $\widehat{Z}$ for multinomial resampling with adaptive decisions is standard [4]. For $\epsilon > 0$ we check that the reweighted resampling step preserves conditional expectations. Let $\mathcal{F}$ be the particle system just before a resampling step, with weights W^k and mean $\bar{W}$. For any f,

$$\mathbb{E}\Big[\bar{W}\,\frac{1}{K}\sum_j \frac{\tilde{w}_{a_j}}{\pi_{a_j}} f(x^{a_j}) \;\Big|\; \mathcal{F}\Big] = \bar{W}\sum_k \tilde{w}_k f(x^k) = \frac{1}{K}\sum_k W^k f(x^k),$$

because $\pi_k > 0$ wherever $\tilde{w}_k > 0$. A resampling step therefore leaves the conditional expectation of the weighted particle measure unchanged, whether or not it fires. Iterating over steps gives $\mathbb{E}[\widehat{Z}] = \mathbb{E}_q[W]$, which equals h_0 by (i).

(iv) At the last step x_T is fully revealed, so $Z_d(x_T; x_{T-1}) = d(x_T)$, and (4) gives $G_T = Z_d(x_{T-1}; x_{T-1})\mathbf{1}_C(x_T)/\big(\psi_{T-1}(x_{T-1})\, d(x_T)\big)$. □

The theorem separates what each kind of decoding needs. Step-exact decoding needs an exact route (Theorem 4.5). Trajectory-exact decoding needs a route whose sampler matches its mass, a twist that is positive on completable states, and an exact membership test at the end. Any NFA or CFG chart, ambiguous or not, meets these conditions in polynomial time per particle. This does not contradict the #P-hardness of computing the exact future-validity function for general CFGs, because SMC never computes h_t; it is exact only in the limit $K \to \infty$.

Proposition 6.3 (Terminal twist [ours])**.** *If an ambiguous route's own value $Z_d(x_T; x_T) = d(x_T)$ is used as the terminal twist in place of $\mathbf{1}_C$, the particles target $\propto p(x_{0:T})\, d(x_T)$, the derivation-weighted law.*

Proof. Replace $\psi_T = \mathbf{1}_C$ by $\psi_T = d$ in the telescoping product of Theorem 6.2(i). □

Remark 6.4 (Relation to prior work)**.** For unambiguous routes, Theorem 6.2 is TWISTER's argument [13] (Thm. 4.2, proof in App. D) applied to a different route. Their proof uses the DFA only to compute Z. Dang et al. [2] (Thm. 4.3 and the remark that follows it) observe, for NFAs in autoregressive SMC, that the derivation-weighted distribution is still a valid proposal. Theorem 6.2 combines these observations for masked diffusion and for every route of Section 5. We have not found the terminal-twist pitfall or the analysis of Section 7 stated elsewhere.

6.4 Choosing the twist separately from the proposal

The proof of Theorem 6.2 uses two facts: the proposal density can be computed, and the twists telescope. The twist can therefore be chosen freely.

Proposition 6.5 (Free intermediate twists [background])**.** *Let M_{t+1} be any kernel with computable density and $M_{t+1}(x' \mid x) > 0$ whenever $\kappa_{t+1}(x' \mid x)h_{t+1}(x') > 0$. Let $\psi_t \geq 0$ with $\psi_t > 0$ where $h_t > 0$, and $\psi_T = \mathbf{1}_C$. With the potentials (1), $qW = p\,\mathbf{1}_C$, and (ii) and (iii) of Theorem 6.2 hold. The same is true for random twists $\psi_t(x, \xi_t)$, where the auxiliary randomness ξ_t is drawn afresh at each step and the same realisation is used in the numerator at step t and the denominator at step $t+1$.*

Proof. The deterministic case is the standard twisted-SMC construction [4]; [16] uses it for autoregressive language models. For random twists, include ξ_t in the state; each realisation appears once in a numerator and once in a denominator, so the product still telescopes path by path.

With the step-exact proposal, the twist used to decide which particles survive can thus be chosen independently of the route used to propose. For example, $\psi_t = \eta_t^\beta$ gives the potential $G_{t+1} = \eta_t(x)^{1-\beta}\eta_{t+1}(x')^\beta / Z_d(x'; x)$, which is the update in Algorithm 2.

6.5 A two-sided identity

Lemma 6.6 ([ours])**.** *Under the conditions of Theorem 6.2,*

$$\mathbb{E}_q[W] = h_0, \qquad \mathbb{E}_p\big[\mathbf{1}_C(x_T)/W(x_{0:T})\big] = \Pr_q(x_T \in C).$$

The second expectation is over native paths. For an exact or derivation-weighted route, $\Pr_q(x_T \in C) = 1$.

Proof. By Theorem 6.2(i), $\mathbb{E}_p[\mathbf{1}_C/W] = \sum_{\text{paths}} p\,\mathbf{1}_C\,q/(p\,\mathbf{1}_C)$, which is the q-probability of ending in C. □

The first identity is checked with proposal runs, the second with native samples. Native paths whose weight exceeds every observed proposal weight point to regions the proposal rarely visits. Running Algorithm 2 with resampling switched off alongside the main run is also useful, because it separates the effect of the proposal and twist from the effect of resampling.

7 Resampling with an over-confident twist

Theorem 6.2(iii) says $\mathbb{E}[\widehat{Z}] = h_0$. If the distribution of $\widehat{Z}$ has most of its expectation in rare events, the average over any feasible number of runs can still fall well short of h_0. This section shows how adaptive resampling with the frozen twist can create such rare events, and gives two remedies. The statements are exact and make no assumption about the denoiser.

7.1 Continuation values and hidden mass

Lemma 7.1 (Continuation value [ours])**.** *Without further resampling, the expected product of the remaining potentials from state x at step s is*

$$V_s(x) := \mathbb{E}_q\Big[\prod_{t\ge s} G_{t+1} \;\Big|\; x_s = x\Big] = \frac{h_s(x)}{\psi_s(x)}.$$

Proof. From step s on, the product of the proposal kernels and the potentials telescopes to $p\,\psi_T/\psi_s$ along every path. Summing over continuations gives $\Pr_p(X_T \in C \mid X_s = x)/\psi_s(x)$. □

Suppose resampling fires at step s, with cumulative weights W_s^k and normalised weights $\tilde{w}_k$. By Theorem 6.2(iii) and Lemma 7.1, any unbiased continuation of the run has

$$\mathbb{E}[\widehat{Z} \mid \mathcal{F}_s] = \sum_k c_k, \qquad c_k := \frac{1}{K} W_s^k \,\frac{h_s(x_s^k)}{\psi_s(x_s^k)}.$$

Write $W_s^k = \rho_s^k\,\psi_s(x_s^k)$, where ρ_s^k is the ratio p/q along particle k's path up to s. Then

$$\tilde{w}_k \propto \rho_s^k\,\psi_s(x_s^k), \qquad c_k \propto \rho_s^k\,h_s(x_s^k). \tag{5}$$

Resampling allocates offspring in proportion to ψ_s, while each particle's share of the expectation is proportional to h_s.

Proposition 7.2 (Hidden mass under multinomial resampling [ours])**.** *Under multinomial resampling ($\pi = \tilde{w}$), particle k has at least one offspring with probability $1 - (1 - \tilde{w}_k)^K \le K\tilde{w}_k$; when it does, its contribution to the run's estimate is multiplied by about $1/(K\tilde{w}_k)$. For $\varepsilon > 0$ define the hidden mass*

$$H_\varepsilon = \frac{1}{h_0}\,\mathbb{E}\Big[\sum_k c_k\,\mathbf{1}\{\tilde{w}_k < \varepsilon\}\Big].$$

Over R independent runs, the expected number of runs in which some particle with $\tilde{w}_k < \varepsilon$ has an offspring is at most $RK\varepsilon\,\mathbb{E}[N_\varepsilon]$, where N_ε is the number of such particles in a run. When this number is much smaller than 1, the average of $\widehat{Z}$ over the R runs misses a fraction of about H_ε of h_0 with high probability, although $\mathbb{E}[\widehat{Z}] = h_0$.

Proof. The offspring count of particle k is binomial with parameters K and $\tilde{w}_k$, which gives the first bound; a union bound over particles and runs gives the second. On the event that particle k has n_k offspring, they carry weight $\bar{W}$ each, against an expected share c_k that Lemma 7.1 assigns to a weight of $W_s^k = K\bar{W}\tilde{w}_k$; hence the factor $n_k/(K\tilde{w}_k)$. □

Tail diagnostics fitted to the observed values of $\widehat{Z}$, such as the generalised-Pareto shape estimate, cannot detect the missing mass, because the events that carry it are absent from the sample.

Corollary 7.3 (A perfect twist hides nothing [ours])**.** *If $\psi_s = h_s$, then $c_k = \tilde{w}_k \sum_j c_j$ by (5), so $H_\varepsilon \le K\varepsilon$.*

Hidden mass is therefore a measure of twist error. It is large when, among the particles of one run, ψ_s understates h_s by orders of magnitude for some of them: the frozen twist is *over-confident* that the constraint has become unreachable.

Remark 7.4 (Estimating H_ε)**.** Runs with resampling switched on and off have the same distribution up to the first step at which resampling would fire. H_ε can therefore be estimated from runs without resampling: record each particle's weight at every step, find the first step at which ESS $< \tau K$, and add up the final contributions of the particles whose normalised weight at that step was below ε.

Remark 7.5 (Why the frozen twist can be over-confident)**.** $\eta_t(x)$ evaluates the constraint under the denoiser's current predictions. Nothing forces those predictions to be consistent with what the denoiser will predict later. A denoiser can assign a completion almost zero probability at one step and a substantial probability after a few more tokens are revealed. h_t averages over these later revisions and η_t ignores them. A synthetic denoiser whose predictions depend little on context has little hidden mass, so tests on such models can miss the effect.

7.2 Defensive resampling

Proposition 7.6 ([ours])**.** *Resample from $\pi = (1 - \epsilon)\tilde{w} + \epsilon/K$ and give each offspring the weight $\tilde{w}_a/\pi_a$, as in Algorithm 2. Then:*

(i) $\mathbb{E}[\widehat{Z}] = h_0$*;*

(ii) every post-resampling weight is at most $1/(1-\epsilon)$, and every particle has at least one offspring with probability at least $1 - (1 - \epsilon/K)^K \ge 1 - e^{-\epsilon}$, however small its weight;

(iii) for any non-negative continuation values g_k, the resampling step adds a conditional relative variance of at most $1/\epsilon$:

$$\frac{\mathrm{Var}\left(\frac{1}{K}\sum_j \frac{\tilde{w}_{a_j}}{\pi_{a_j}} g_{a_j} \mid \mathcal{F}\right)}{(\sum_k \tilde{w}_k g_k)^2} \le \frac{1}{\epsilon}\,\frac{\sum_k (\tilde{w}_k g_k)^2}{(\sum_k \tilde{w}_k g_k)^2} \le \frac{1}{\epsilon}.$$

Under multinomial resampling the same ratio is $\frac{1}{K}\big(\sum_k \tilde{w}_k g_k^2/(\sum_k \tilde{w}_k g_k)^2 - 1\big)$, which has no bound when a particle with small $\tilde{w}_k$ has a large g_k.

Proof. (i) is Theorem 6.2(iii). (ii) follows from $\pi_k \ge (1-\epsilon)\tilde{w}_k$ and $\pi_k \ge \epsilon/K$. For (iii), let $Y = (\tilde{w}_a/\pi_a)\, g_a$ with $a \sim \pi$. Then $\mathbb{E}Y = \sum_k \tilde{w}_k g_k$ and $\mathbb{E}Y^2 = \sum_k \tilde{w}_k^2 g_k^2/\pi_k \le (K/\epsilon)\sum_k (\tilde{w}_k g_k)^2$. The variance of an average of K independent copies is at most $\mathbb{E}Y^2/K$, and $\sum_k (\tilde{w}_k g_k)^2 \le (\sum_k \tilde{w}_k g_k)^2$ because $g \ge 0$. □

Taking $g_k = V_s(x^k)$ (Lemma 7.1) covers the situation of Proposition 7.2. The construction is the resampling counterpart of defensive importance sampling [5]. When the twist is accurate it costs some resampling efficiency, and in exchange its variance bound holds whatever the twist's errors.

7.3 Tempered intermediate twists

Proposition 7.7 ([ours])**.** *For $\beta \in (0,1]$, let $\psi_t = \eta_t^\beta$ for $t < T$ and $\psi_T = \mathbf{1}_C$, with the step-exact proposal. The potential is $G_{t+1} = \eta_t(x)^{1-\beta}\eta_{t+1}(x')^\beta / Z_d(x';x)$, and (ii) and (iii) of Theorem 6.2 hold. In (5), $\tilde{w}_k \propto \rho_s^k\, \eta_s(x_s^k)^\beta$, so*

$$\frac{c_k}{\tilde{w}_k} \propto h_s(x_s^k)^{1-\beta}\Big(\frac{h_s(x_s^k)}{\eta_s(x_s^k)}\Big)^\beta.$$

Proof. $\eta_t^\beta > 0$ wherever $\eta_t > 0$, which includes every completable state, so Proposition 6.5 applies. The formula for $\tilde{w}_k$ is (5) with $\psi_s = \eta_s^\beta$. □

An over-confidence factor $h_s/\eta_s = \delta^{-1}$ thus enters resampling as $\delta^{-\beta}$. The price is that resampling uses only part of the information in the frozen twist. At $\beta \to 0$, particles are resampled on the path ratio ρ alone; $\beta = 1$ is TWISTER. The two remedies can be combined.

Remark 7.8 (When tempering is enough)**.** Tempering removes hidden mass only if the tempered mismatch is large enough to be sampled. By Proposition 7.2, a particle whose normalised weight is reduced by a factor δ^β relative to its share of the expectation has an offspring in some run out of R only if $RK\delta^\beta \gtrsim 1$, which requires $\beta \lesssim \log(RK)/\log(1/\delta)$. When the frozen twist is over-confident by many orders of magnitude, this forces β close to 0 and discards most of the look-ahead. Defensive resampling has no such condition, because its retention probability $1 - e^{-\epsilon}$ does not depend on δ. It is therefore the safer default; tempering is a complement for moderate twist errors.

8 Ambiguity

By Theorem 6.2, an ambiguous route gives the correct target. It costs variance in two ways. The proposal favours completions with many derivations, and the twist $\eta_d = \sum_y d(y)q(y)$ overstates how reachable C is, in proportion to the ambiguity of the remaining completions.

Proposition 8.1 (Single-step cost of ambiguity [ours]). *Suppose* $1 \le d(y) \le D$ *on* C. *Importance sampling from* $q_d(y) \propto d(y)q(y)\mathbf{1}_C(y)$ *to* $q(y \mid C)$ *uses weights proportional to* $1/d(y)$, *and its ESS fraction satisfies*

$$\frac{(\mathbb{E}_{q_d}[1/d])^2}{\mathbb{E}_{q_d}[1/d^2]} \ge \frac{4D}{(1+D)^2}.$$

The twist ratio satisfies $1 \le \eta_d/\eta \le D$.

Proof. Let $X = 1/d(y)$ with $y \sim q_d$, so $X \in [m, M]$ with $m = 1/D$ and $M = 1$, and write $\mu = \mathbb{E}X$. From $(X - m)(M - X) \ge 0$ we get $X^2 \le (m + M)X - mM$, hence $\mathbb{E}X^2 \le (m + M)\mu - mM$. The quadratic $\frac{(m+M)^2}{4mM}\mu^2 - (m + M)\mu + mM$ has discriminant zero, so it is non-negative, which gives $\mathbb{E}X^2 \le \frac{(m+M)^2}{4mM}\mu^2$ (Kantorovich's inequality). Substituting $m = 1/D$ and $M = 1$ gives the ESS bound. The twist bound is immediate from $1 \le d \le D$. □

The bound is informative when the ambiguity is bounded. For the RNA pairing grammar, D grows exponentially with L and the bound is vacuous. Early in decoding, though, the twist only sees the ambiguity of completions consistent with the current partial sequence, which can be much smaller than D. Whether a per-step measure of this kind controls the ESS of whole trajectories is open (Section 10).

Exact counting that ignores derivations is out of reach for ambiguous grammars in general. For CFG constraints on hidden Markov models, the exact constrained likelihood is NP-hard already at ambiguity 2, and an FPRAS exists when the ambiguity is polynomially bounded [9]. This suggests approximate twists under bounded ambiguity. By Proposition 6.5, approximate or randomised twists are allowed, provided the proposal density is exact and the terminal twist is $\mathbf{1}_C$.

9 Beyond exact routes

9.1 A regular superset with Boolean pruning [ours]

For a CFG constraint C, choose a DFA for a regular superset $R \supseteq C$: the lexer or class-level DFA, bracketing up to a depth bound with an overflow state that accepts everything, or the regular part of a schema. Use the chain route for R as the proposal, with twist

$$\psi_t(x) = Z_R(x; x) \cdot \mathrm{comp}_C(x), \qquad \psi_T = \mathbf{1}_C,$$

where $\mathrm{comp}_C(x) \in \{0, 1\}$ reports whether x can be completed to an element of C (an emptiness test for the intersection of a context-free and a regular language). By Proposition 6.5 this is exact provided

1. $C \subseteq R$, so the proposal reaches every completable state, and
2. comp_C never reports 0 for a completable state. Reporting 1 for a dead state only wastes a particle.

Exact completability checkers satisfy condition 2. A checker that accepts a state only if one of a few sampled fillings can be extended may reject completable states, and then biases the target. A regular *subset* (an overflow state that rejects) violates condition 1 and is biased unless the depth bound provably covers every string of length L. Pruning turns one accept-or-reject decision at the end into deaths at individual steps, which resampling can repair. R should be tight enough that the per-step death rate stays well below $1 - 1/K$.

Redrawing [proposed]. Instead of letting a proposal that lands on a dead state kill its particle, redraw until comp_C accepts. The potential then needs the acceptance probability $a(x)$ of a single draw. Using an unbiased estimate of $a(x)$ (the accepted fraction among m extra draws) keeps the algorithm valid by the random-twist part of Proposition 6.5.

9.2 Conjunctions

Conjunctions of automaton constraints compile to product automata, whose size is the product of the factors' sizes. A CFG intersected with a regular language is context-free, by the Bar-Hillel construction, with a grammar polynomial in both. A copy relation conjoined with local constraints stays a factor graph as long as the combined width stays bounded. A determinant route conjoined with other constraints does not fit any single route described here. A conjunction whose counting problem is #P-hard is refused.

9.3 The intractable boundary

If a constraint family has no FPRAS, it has no polynomial-time almost-uniform sampler either [7]. The trajectory layer can still be run with a superset route and a weaker twist, but its variance can grow exponentially with L. Refusing such constraints is the correct behaviour for the compiler.

10 Open directions

1. *Better twists.* Proposition 7.2 makes the over-confidence of the frozen twist the main practical limit, and Proposition 6.5 leaves the intermediate twists free. Candidates:
   (a) tempering and defensive resampling (Propositions 7.6 and 7.7), which are cheap and come with guarantees that do not depend on the twist;
   (b) look-ahead twists, which evaluate Z under the predictions made after one or more speculative reveals, at the cost of extra denoiser calls;
   (c) mixture twists $\psi_t = (1-\lambda)\eta_t + \lambda\hat{h}_t$, where $\hat{h}_t$ is an unbiased estimate of h_t from native rollouts and the η_t term keeps the twist positive;
   (d) learned twists $\psi_\vartheta \approx h_t$ fitted to native rollouts, as in twisted SMC for autoregressive models [16], with η_t as a feature or baseline;
   (e) approximate-count twists for bounded ambiguity [9].

2. *Mixing the native kernel into the proposal.* $M_\alpha = (1-\alpha)M + \alpha\kappa$ has a computable density, and $G \le \psi_{t+1}/(\alpha\psi_t)$ for every step. It reaches regions the step-exact proposal rarely visits, at the price of proposals that leave C and get weight zero.

3. *Trajectory ESS under ambiguity.* Bound the ESS of whole paths in terms of a per-step ambiguity measure, such as the spread of d over completions consistent with the current partial sequence, instead of the global range D in Proposition 8.1.

4. *An online guard against hidden mass.* Proposition 7.2 expresses the hidden mass through h_s/ψ_s, which cannot be observed. An estimate or upper bound available during a run (for example, from a small population of particles that is never resampled) would turn the offline diagnostic into a check made while decoding.

5. *Routes for RNA and mRNA design.* A route for a fixed target structure (a tree-shaped factor graph over paired and unpaired positions) and charts with a bound W on pair span (cost

$O(LW^2|G|)$). An external energy model can enter through the twist, and Proposition 6.5 covers this without change.

6. *Calibrated sampling and best-of-K.* Exact conditional sampling should eventually beat mode-seeking decoding in best-of-K selection as K grows. Where the crossover lies is open.

11 Summary of results

result	status	source or note
Semirings and ambiguity (Proposition 4.3); MAP for any CFG	background	[14]
Circuit characterisation (Theorem 4.5)	background	[1, 7]
Spanning trees separate monotone from general circuits (Remark 4.7)	background	[6]
Step-exact proposal (Lemma 6.1); free twists (Proposition 6.5)	background	[13, 4]
Routes with single-pass exact samplers; compiler with refusal	ours (system)	Section 5
Route-generic trajectory exactness (Theorem 6.2)	ours	extends [13]; compare [2]
Terminal-twist pitfall	ours	
Two-sided identity (Lemma 6.6)	ours	diagnostic
Continuation value and hidden mass (Lemma 7.1 and Proposition 7.2)	ours	
Defensive resampling (Proposition 7.6)	ours	compare [5]
Tempered intermediate twists (Proposition 7.7)	ours	
Single-step cost of ambiguity (Proposition 8.1)	ours	
Regular superset with Boolean pruning	ours	Section 9
Better twists; trajectory ESS; online guard	proposed	Section 10

References

[1] O. Broadrick, S. Agarwal, G. Van den Broeck and M. Bläser. The limits of tractable marginalization. arXiv:2506.12020, 2025.

[2] M. Dang, J. Song, H. Zhang, H. Zhao, G. Van den Broeck and S. Ermon. Mitigating bias in locally constrained decoding via tractable proposals. ICML 2026; arXiv:2606.01926.

[3] M. Dang and S. Ermon. Constrained decoding for diffusion language models via efficient inference over finite automata. arXiv:2607.07026, 2026.

[4] P. Del Moral. *Feynman–Kac Formulae: Genealogical and Interacting Particle Systems with Applications*. Springer, 2004. doi:10.1007/978-1-4684-9393-1.

[5] T. Hesterberg. Weighted average importance sampling and defensive mixture distributions. *Technometrics* 37(2), 1995. doi:10.1080/00401706.1995.10484303.

[6] M. Jerrum and M. Snir. Some exact complexity results for straight-line computations over semirings. *J. ACM* 29(3), 1982. doi:10.1145/322326.322341.

[7] M. R. Jerrum, L. G. Valiant and V. V. Vazirani. Random generation of combinatorial structures from a uniform distribution. *Theor. Comput. Sci.* 43:169–188, 1986. doi:10.1016/0304-3975(86)90174-X.

[8] T. Koo, A. Globerson, X. Carreras and M. Collins. Structured prediction models via the matrix-tree theorem. In *Proc. EMNLP-CoNLL 2007*, pp. 141–150. ACL Anthology D07-1015.

[9] R. Marzouk and C. de la Higuera. Marginal inference queries in hidden Markov models under context-free grammar constraints. arXiv:2206.12862, 2022.

[10] R. Nussinov and A. B. Jacobson. Fast algorithm for predicting the secondary structure of single-stranded RNA. *PNAS* 77(11), 1980. doi:10.1073/pnas.77.11.6309.

[11] S. S. Sahoo, M. Arriola, Y. Schiff, A. Gokaslan, E. Marroquin, J. T. Chiu et al. Simple and effective masked diffusion language models. arXiv:2406.07524, 2024.

[12] J. Su and W. Zhang. Constraints are graphs, not chains: exact decoding for diffusion language models. arXiv:2609.32900, 2026.

[13] A. Thimmaiah, L. Marinov, J. Srinivasa, H. Vikalo, J. J. Li and M. Gligoric. Twist, don't tilt: trajectory-exact constrained decoding for masked diffusion models. arXiv:2609.35609, 2026.

[14] B. Wang, D. D. Mauá, G. Van den Broeck and Y. Choi. A compositional atlas for algebraic circuits. arXiv:2412.05481, 2024.

[15] D. B. Wilson. Generating random spanning trees more quickly than the cover time. In *Proc. STOC 1996*. doi:10.1145/237814.237880.

[16] S. Zhao, R. Brekelmans, A. Makhzani and R. Grosse. Probabilistic inference in language models via twisted sequential Monte Carlo. arXiv:2404.17546, 2024.